

# Sécurité avancée des SI d'entreprise

Authentication

---

Daniel De Almeida Braga

**How do the OS authenticates users?**

**How do the OS authenticates users?**

**You have 1h30.**

# Introduction



# What is Authentication?

---

- Proving the identity of an entity (user, machine, software, ...)
- Binds an identity to an entity
- Can happen locally or remotely
- 2 steps:
  - Sign up: Defines the identity binding on the system
  - Sign in: Verifies and binds the identity of the entity

# The Concept

---

- The authenticator (e.g. server, website) asks to prove that you are who you pretend to be based on one or more pieces of evidence called factors.
  - Mutual authentication is also a thing
- Different types of factors
  - What you know (password),
  - What you have (phone, smart card, token),
  - What you are (biometrics),
  - ...

# Muli-Factor Authentication (MFA)

---

## Concepts and Advantages of MFA

- Chaining multiple factors to enhance security.

## Strong Authentication

- At least 2 factors (2FA)...
- ...that should not be easily bypassed or broken

# What You Know - Password Authentication

---

## Examples

- Usernames and passwords for online accounts.
- PINs for ATM transactions.

## Advantages

- Simple and easy to use.
- Low implementation cost.
- Widely understood by users.

## Drawbacks

- Vulnerable phishing, brute force, and dictionary attacks.
- Users often choose weak passwords.
- Passwords can be forgotten or stolen.



# Password Authentication - Pitfalls and Recommendations

---

## Pitfalls

- Relying solely on passwords without additional security measures.
- Not enforcing strong password policies.
- Storing passwords in plaintext.

## Recommendations

- Implement MFA.
- Enforce strong password policies, including complexity requirements.
- Store passwords using a good hash function and salt.

# What You have - Token-Based Authentication

---

## Examples

- Hardware tokens (e.g., RSA SecurID, YubiKey, Google Titan, FIDO2 keys...).
- Software tokens (e.g., Google Authenticator).
- Smart cards.

## Advantages

- Higher security than passwords alone.
- Reduces the risk of phishing attacks.

## Drawbacks

- Can be lost, stolen, or damaged.
- Additional cost for hardware tokens.
- Requires user to carry an additional device.

# Token-Based Authentication - Pitfalls and Recommendations

---

## Pitfalls

- Not properly securing token distribution and management.
- Relying solely on tokens without additional security measures.
- Inadequate backup mechanisms for lost or stolen tokens.

## Recommendations

- Implement robust procedures for token issuance and management.
- Combine token-based authentication with other methods (e.g., passwords) for MFA.
- Provide users with a way to recover from lost or stolen tokens securely.

# What You Are - Biometric Authentication

---

## Examples

- Fingerprint recognition.
- Facial recognition.
- Iris scanning.
- Voice recognition.



## Advantages

- Difficult to forge or steal.
- Convenient for users (no need to remember passwords or carry tokens).
- Can provide high levels of security.

## Drawbacks

- Privacy concerns (biometric data is sensitive).
- Potential for false positives/negatives.
- High implementation cost.

# Biometric Authentication - Pitfalls and Recommendations

---

## Pitfalls

- Storing biometric data without proper encryption.
- Not considering user consent and privacy regulations.
- Over-reliance on a single biometric trait.

## Recommendations

- Encrypt biometric data and store it securely.
- Ensure compliance with privacy laws and obtain user consent.
- Use multiple biometric traits to improve accuracy and security.

# Authentication on Linux

---

# Basic Authentication Mechanisms on Linux

---

- **Configuration Files**

- `/etc/passwd`: Contains user information.
- `/etc/shadow`: Contains hashed passwords.

- **Commands**

- `passwd`: Change user password.
- `useradd`, `usermod`, `userdel`: Manage users.

## Example Command

```
sudo useradd -m student1  
sudo passwd student1
```

# Password Storage in /etc/shadow

---

- /etc/shadow file stores hashed passwords.

- **Format**

username:password:lastchange:min:max:warn:inactive:expire:flag

- **Example Entry**

user:\$6\$salt\$hashed\_password:18000:0:99999:7:::

- **Hashing Algorithms**

- <nothing> : DES
- \$1\$: MD5
- \$5\$: SHA-256
- \$6\$: SHA-512
- \$y\$: yescrypt



# Network Authentication - SSH

---

- **Generating and Using SSH Keys**

- Generate a key pair: `ssh-keygen -t ed25519`
- Public key: `~/.ssh/id_ed25519.pub`
- Private key: `~/.ssh/id_ed25519`

- **Configuring Keys**

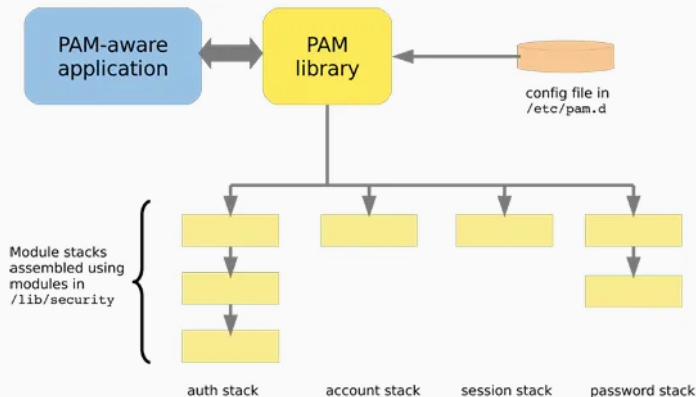
- Copy the public key to the server: `ssh-copy-id user@server`
- Public key added to: `~/.ssh/authorized_keys`

- **Securing the SSH Service**

- Configuration file: `/etc/ssh/sshd_config`
- Disable password authentication: `PasswordAuthentication no`
- Add a proper PAM configuration

# PAM (Pluggable Authentication Modules)

- PAM manages authentication through configurable modules.
- Each service has its own configuration file in `/etc/pam.d/`.
- 4 management groups (categories)



## PAM - Configuration Structure

---

Each configuration file contains a set of **rules** with the following structure

```
type control module [arguments]
```

# PAM - Configuration Structure

---

Each configuration file contains a set of **rules** with the following structure

```
type control module [arguments]
```

- **type**: Management group of the module (account, session, auth, password)
- **control**: Defines the behavior of PAM:
  - **required**: Must succeed, but if a single module fails, still check the other modules.
  - **requisite**: Must succeed, immediately exit on failure.
  - **sufficient**: Can fail if there is at least another sufficient module that succeeds. No additional checks if it succeeds.
  - **optional**: Can fail without consequence.
- **module**: Name of module.
- **arguments**: Arguments that passed for the module.

## PAM - Common Modules

---

- `pam_unix.so`: Standard Unix authentication (`/etc/shadow`).
- `pam_tally2.so`: Lockout after a number of failed login attempts.
- `pam_{permit, deny}.so`: Always {succeeds, fails}.
- `pam_nologin.so`: Allows blank password.
- `pam_pwquality.so`: Perform password quality checking (previously, `pam_cracklib`).
- `pam_rootok.so`: Succeeds if you are root.

## PAM - What does this configuration do?

---

```
auth sufficient pam_permit.so
```

## PAM - What does this configuration do?

---

```
auth sufficient pam_permit.so
```

**Login without any checks!**

## PAM - What does this configuration do?

---

```
auth required pam_tally2.so deny=3 unlock_time=1200
```



## PAM - What does this configuration do?

---

```
auth required pam_tally2.so deny=3 unlock_time=1200
```

**Lock the account for 20min after 3 attempts**

## PAM - What does this configuration do?

---

### `/etc/pam.d/common-auth`

```
auth [success=1 default=ignore] pam_unix.so nullok
auth requisite pam_deny.so
auth required pam_permit.so
auth optional pam_cap.so
```

## PAM - What does this configuration do?

---

### `/etc/pam.d/common-auth`

```
auth [success=1 default=ignore] pam_unix.so nullok
auth requisite pam_deny.so
auth required pam_permit.so
auth optional pam_cap.so
```

- Default to Unix authentication (allow blank password)
- Fallback if the previous module failed
- Set a success
- Set the current process capabilities

# Multi-Factor Authentication (MFA)

---

## Implementing Google Authenticator

- Install Google Authenticator: `sudo apt-get install libpam-google-authenticator`
- Configure PAM: more details in the next Practical Session!

# User Identification

---

- Each user is assigned a unique User ID (UID).
- **UID 0**: Reserved for the root user (superuser).
- **UIDs 1-99**: Reserved for system and administrative accounts.
- **UIDs 100-999**: Reserved for system accounts and services.
- **UIDs 1000 and above**: Typically assigned to regular users.
- UID is stored in `/etc/passwd`.

# Group Management in Linux

---

- Groups are used to manage permissions for multiple users.
- `/etc/group` file contains group information.
- **Primary Group**
  - Each user is assigned a primary group.
  - Primary group is specified in `/etc/passwd`.
- **Supplementary Groups**
  - Users can belong to multiple supplementary groups.
  - Manage supplementary groups with `usermod -aG groupname username`.
- **Example Entry in `/etc/group`**

`groupname:x:1001:user1,user2`

# Authentication on Windows

---

# Basic Authentication Mechanisms on Windows

---

- **User and Group Management**

- Graphical Interface: `lusrmgr.msc`
- Commands (cmd): `net user`, `net group`
- Commands (PowerShell): `New-LocalUser`, `New-AdGroup`, `New-AdUser`, `New-AdGroup`

- **Password Policy and Account Lockout**

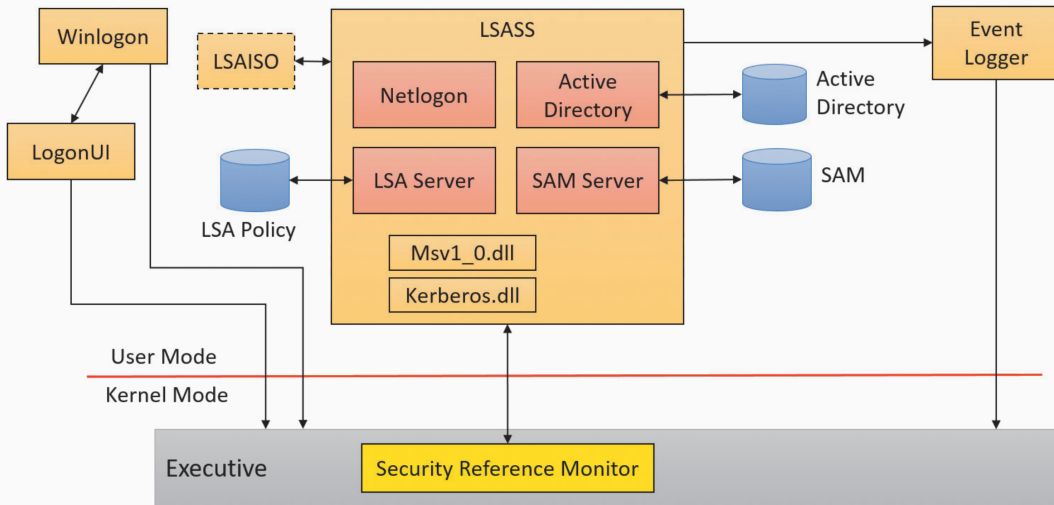
- Configuration tool: `secpol.msc`, `gpedit.msc`
- Configure password and account lockout policies.

## Example Command

```
net user student1 /add  
net localgroup Administrators student1 /add
```



# Authentication Components 1/2



# Authentication Components 1/2

---

- **Winlogon:** Interactive user authentication through LSASS
- **Credential Provider:** In-process objects (in LogonUI) used to get authentication factors.
- **Security Reference Monitor (SRM):** Defines the access token data structure to represent a security context, performing security access checks on objects, manipulating privileges (user rights), and generating any resulting security audit messages.

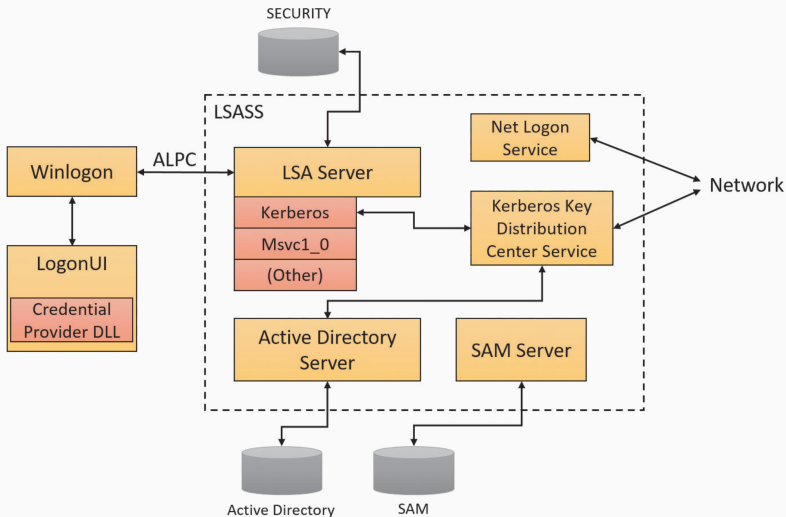
## Authentication Components 2/2

---

### Local Security Authority Subsystem Service (LSASS)

- **LSA:** User-mode process responsible for the local system security policy
- **Security Account Manager (SAM):** Database that stores local user accounts and security descriptors (including hashed passwords).
- **Credential Guard:** Store users' token hashes instead of keeping them in Lsass's memory
- **Netlogon:** Service used to set up a secure channel with domain controller
- **Active Directory:** Service that contains information about objects in domains
- **Authentication Packages:** Dynamic library implementing Windows authentication policy

# Local Authentication Workflow



# Local Password-based Authentication

---

Uses MSV1\_0 Authentication Packages

- Default authentication package on a standalone Windows
- Takes the username and a hashed version of the password
- Query the local SAM to retrieve the account information
- Can be used in old (before Windows 2000) domains

# Local Password-based Authentication

---

Uses MSV1\_0 Authentication Packages

- Default authentication package on a standalone Windows
- Takes the username and a hashed version of the password
- Query the local SAM to retrieve the account information
- Can be used in old (before Windows 2000) domains

## (NT)LM hashing

- **No salt, and very weak hash function**
- LAN Manager (LM) hash are deprecated.
- NTLMv2 is based on MD4...
- No anti-replay mechanism
- Most local authentication use this

## Network Authentication - NTLM protocol

---

Also provided by MSV1\_0 AP

- Challenge-response between client and DC/server
- Still supported, but Kerberos is preferred
- Different security based on *LM compatibility level* (0 - 5)
- Vulnerable to pass-the-hash attacks

# SMB NTLMv2 challenge-response authentication protocol (simplified)

### SMB\_NEGOTIATE\_PROTOCOL\_REQUEST

includes supported dialects & flags



### SMB\_NEGOTIATE\_PROTOCOL\_RESPONSE

Agrees on dialect to use & flags

includes **8-byte server challenge/nonce** (C)



### SMB\_SESSION\_SETUP\_ANDX\_REQUEST

includes username, domain

**24-byte LMv2** =  $\text{hmac\_md5}(\text{ntv2hash}^*, \text{server\_nonce} + \text{client\_challenge}) + 8\text{-byte client\_challenge}$

**16-byte NTv2** =  $\text{hmac\_md5}(\text{ntv2hash}^*, \text{server\_nonce} + \text{blob}^{**})$

**8-byte TimeStamp**

**8-byte client\_challenge** (yes, again..)

\* $\text{ntv2hash\_server} = \text{hmac\_md5}(\text{nt\_hash}, \text{unicode}(\text{upper}(\text{user})) + \text{unicode}(\text{upper}(\text{domain})))$

\*\* $\text{blob} = (\text{TimeStamp} + \text{client\_challenge} + \text{domain} + \text{data})$



### SMB\_SESSION\_SETUP\_ANDX\_RESPONSE

**Allows or disallows access**



Calculates LMv2 and/or NTv2,  
compares result with client  
response





## NTLM protocol - Compatibility Settings

---

LM compatibility level is defined as a group policy in  
GPO\_name\Computer Configuration\Windows Settings\Security Settings\Local  
Policies\Security Options

- 0 Send LM & NTLM responses
- 1 Send LM & NTLM - use NTLMv2 session security if negotiated
- 2 Send NTLM response only
- 3 Send NTLMv2 response only (default)
- 4 Send NTLMv2 response only. Refuse LM
- 5 Send NTLMv2 response only. Refuse LM & NTLM

# Network Authentication - Kerberos

---

Uses Kerberos Authentication Packages

- Default on Active Directory
- Based on a trusted 3rd party for *mutual* authentication
- Authentication is based on the attribution of *tickets*
- *Tickets* can then be used to authenticate to a suite of network services
- Password (or hashes) are not transmitted through the network

## 3 components:

- A client trying to authenticate
- A server/DC
- The trusted 3rd party: a Key Distribution Center (KDC)

**In practice:** The KDC is usually the domain controller

# Kerberos Vocabulary

---

- Computers managed by the same KDC constitute a kerberos *real*
- Users, clients and servers are called kerberos *principals*
- A *ticket* is a data structure containing an encrypted and a clear part. Tickets are signed by the KDC, and have a limited validity.
  - Ticket Granting Ticket (TGT)
  - Service Ticket (ST)

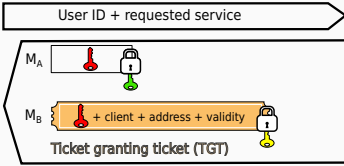


Client (C)



$K_{C-TGS}$

### Client Authentication to the AS



Key Distribution  
Center (KDC)



$K_C$   
 $K_{TGS}$

Authentication  
Server (AS)



$K_{TGS}$   
 $K_S$

Ticket-granting  
Server (TGS)



$K_S$

Service  
Server (SS)



Client (C)

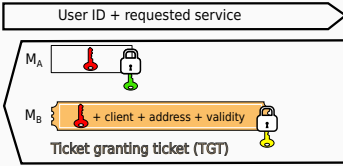


$K_{C-TGS}$

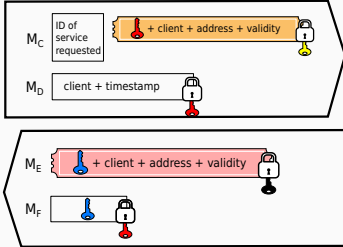


$K_{C-S}$

### Client Authentication to the AS



### Client Service Authorization



Key Distribution Center (KDC)



Authentication Server (AS)



Ticket-granting Server (TGS)



Service Server (SS)





Client (C)

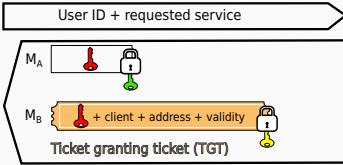


$K_{C-TGS}$

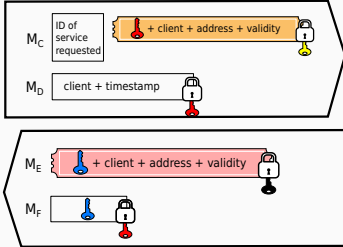


$K_{C-S}$

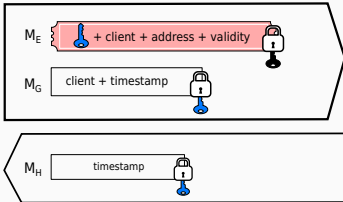
### Client Authentication to the AS



### Client Service Authorization



### Client Service Request



### Key Distribution Center (KDC)



Authentication Server (AS)

$K_C$

$K_{TGS}$



Ticket-granting Server (TGS)

$K_{TGS}$

$K_S$



Service Server (SS)

$K_S$

# Multi-Factor Authentication (MFA)

---

Supported authentication factors are provided by credential providers

- `authui.dll`: password
- `SmartcardCredentialProvider.dll`: smart card PIN
- `FaceCredentialProvider.dll`: face-recognition authentication
- NGC Credential Provider: PIN of Windows Hello
- FIDO Credential Provider: FIDO2 security keys
- ...

Available providers are listed at the key

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Authentication\Credential Providers`

## Facial/Iris or fingerprint scanning

- Risk of false acceptance (1/100 000 says Microsoft)
- Risk of false rejection (e.g., abnormal lighting): 1%
- PIN as a fallback

## PIN authentication

- Stored locally in the TPM



## Other solutions

---

### Authenticator App

- Push notification/TOTP on a phone
- Require a phone
- Relies on biometric or PIN authentication on the phone

### FIDO2 security key

- Require a hardware token
- FIDO2 keys can require biometric, PIN or NFC

### Certificate-based authentication

- Easy to deploy and manage
- Tweakable policy with certificate fields (issuer, OID, ...)
- May not scale very well (e.g. if you use multiple keys for each user)

# Identity Management on Windows

---

- Windows uses Security Identifiers (SIDs) to manage identities.
- SIDs are unique identifiers assigned to each user, group, and computer account.
- SIDs are generated by the Local Security Authority (LSA) when accounts are created.
- SIDs are immutable and persist even if the account is renamed.
- Tools for managing SIDs:
  - `whoami /user`: Display the SID of the current user.
  - `wmic useraccount get name,sid`: List all user accounts and their SIDs.
  - `icacls`: Modify and view ACLs containing SIDs.

# SID Format

---

- SIDs have a specific structure:

S-1-5-21-xxxxxxxxxx-xxxxxxxxxx-xxxxxxxxxx-xxxx

- **Components of SID**

- **S-1:** Identifier authority (Security Identifier Version).
  - **5:** Identifier authority value (NT Authority).
  - **21:** Domain or local computer identifier.
  - **xxxxxxxxxx-xxxxxxxxxx-xxxxxxxxxx:** Sub-authority values (e.g., domain or local computer identifier).
  - **xxxx:** Relative Identifier (RID) specific to the user or group.
- Example SID: S-1-5-21-3623811015-3361044348-30300820-1013

## Well-Known SIDs

---

- Well-Known SIDs are predefined SIDs for commonly used security principals.
- Well-Known SIDs are consistent across all installations of Windows.
  - **S-1-0-0**: Nobody (SID unknown)
  - **S-1-1-0**: Everyone (group all users except anonymous)
  - **S-1-5-18**: Local System Account
  - **S-1-5-19**: Local Service Account
  - **S-1-5-20**: Network Service Account
  - **S-1-5-32-544**: Administrators Group
  - **S-1-5-32-545**: Users Group
  - **S-1-5-32-546**: Guests Group

[More well-known sid](#)

## Successful login

---

Upon successful login, LSASS create an initial *token* that identify the *security context*

- SID
- Groups membership
- Integrity level
- Privileges
- ...

This token will be inherited by all process spawned by the user